

# Clustering Analysis of Domestic Water Usage Across Student Housing Types: Evidence from the University of Texas at Austin

Tianlu Zhang

2025-08-10

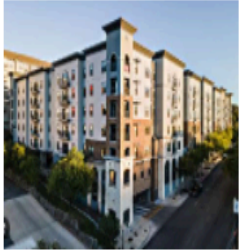
## 1. Background & Introduction

### 1.1 Background

This report is part of the Austin Water & University of Texas at Austin CONNECT 2025 Summer Project. Tianlu Zhang, a Ph.D. student at the University of Texas at Austin, received a fellowship through this program and worked with Austin Water for three months as a data science consultant on a student housing water usage benchmarking project. This document is part of the final report.

This project contributes to a broader initiative aimed at benchmarking water usage in student housing to inform future water conservation efforts. The objective of this phase is to examine whether different categories of student housing exhibit distinct water usage patterns, particularly over the most recent three-year period.

The analysis uses longitudinal Advanced Metering Infrastructure (AMI) data at the property level, covering **ten student housing properties** (see picture below) near the UT Austin campus and spanning back to 2015. These properties were classified into four types based on building characteristics (e.g., construction year, scale, unit design) and resident composition. An unsupervised machine learning approach (DBSCAN) was applied to identify seasonal water usage patterns within each housing category, and the results were compared across categories to assess differences in consumption trends across seasons and years.

| Type                | Type 1: Dorm/Resident hall                                                        | Type 2: Graduate student housing                                                  | Type 3: Shared-Occupancy Apartments                                                | Type 4: Established Large-Scale Apartments (no shared Occupancy)                    |
|---------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Features            | Resident hall/dorm style apartment outside main UT campus                         | Student housing provided by UT specifically to graduate students                  | Aprtments with 100+ units, built for 10-20 years, availability of shared rooms     | 100+ units built 10-20 years                                                        |
| What it looks like? |  |  |  |  |
| Apartment name      | Callaway-house<br>Castilian                                                       | UT East graduate apartments<br>Colorado Apartments                                | The 2400 Nueces apartments                                                         | Crest At Pearl<br>The Block on 28th<br>26 West<br>Texan<br>Vintage<br>The_Block     |

## 1.2 Hypothesis

Based on prior research, it was hypothesized that residence hall/dormitory style housing and established large-scale shared-occupancy apartments would display stronger seasonal variation, with notably lower consumption during summer and winter breaks, whereas graduate student housing would demonstrate less seasonal fluctuation and more stable usage throughout the year.

## 1.3 Analyses plan

For each housing type, the mean water consumption of all properties within that category was calculated for **each month** to represent the monthly usage of the housing type from November 2015 to June 2025.

Building on prior research that uses domestic cold water (DCW) data as a proxy for occupancy (Fu et al., 2021), an unsupervised machine learning clustering method, **Density-Based Spatial Clustering of Applications with Noise** (DBSCAN; Ester et al., 1996) was applied to monthly water consumption data, separately for each of the four student housing types. DBSCAN identifies clusters by selecting a data point and counting how many neighboring points (MinPts is the minimum number of points required) fall within a specified distance ( $\epsilon$ ). If the number of neighbors exceeds a user-defined threshold, the point is assigned to a cluster, and the cluster is expanded iteratively. Points that cannot be assigned to any cluster are classified as outliers.

To determine the optimal distance  $\epsilon$ , the k-nearest neighbor (kNN) method was used (Chen et al., 2021). For each data point, the distance to its k-th nearest neighbor was calculated, and these distances were then sorted in ascending order. Plotting the sorted kNN distances produces a curve that typically shows a sharp change in slope. The location of this change, referred to as the elbow point, was selected as  $\epsilon$ , as it represents a value large enough to include the densest meaningful clusters while remaining small enough to exclude noise. DBSCAN was implemented in R using the package developed by Hahsler et al. (2019).

## 2.Overall conclusion

### Key takeaways

Different types of student housing exhibited distinct water usage patterns, specifically:

1. **Residence hall/dormitory style:** Water usage showed clear seasonal reductions, with notably lower consumption during summer (May-August) and December, aligning with the academic calendar.
2. **Graduate student housing:** Usage remained relatively stable across the year but declined significantly during the COVID-19 pandemic, with a slower recovery compared to other housing types.
3. **Large-scale shared-occupancy apartments:** Seasonal reductions were concentrated in winter (December–January), reflecting usage patterns distinct from dormitories due to mixed unit structures.
4. **Large-scale apartments without shared occupancy:** This housing type displayed unusually high usage peaks in September and October prior to 2020, likely linked to social activities, followed by a pronounced post-pandemic decline in overall consumption.

**Future efforts to establish water usage benchmarks (gallons per person) for student housing should account for differences across housing types driven by resident characteristics, as well as seasonal and temporal variations.**

## 3.Data wrangling

### 3.1 Import packages

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
## ✓ forcats    1.0.1      ✓ readr      2.1.5
## ✓ ggplot2    4.0.2      ✓ stringr    1.5.2
## ✓ lubridate  1.9.4      ✓ tibble     3.3.0
## ✓ purrr      1.1.0      ✓ tidyr      1.3.1
```

```
## — Conflicts ————— tidyverse_conflicts() —
## * dplyr::filter() masks stats::filter()
## * dplyr::lag()      masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
library(bruceR)
```

```
##
## bruceR (v2025.8)
## Broadly Useful Convenient and Efficient R functions
##
## Packages also loaded:
## ✓ dplyr      ✓ data.table
## ✓ tidyr      ✓ emmeans
## ✓ stringr    ✓ lmerTest
## ✓ forcats    ✓ effectsize
## ✓ ggplot2    ✓ performance
## ✓ cowplot    ✓ interactions
##
## Main functions of `bruceR`:
## cc()          Describe()  TTEST()
## add()          Freq()      MANOVA()
## .mean()        Corr()      EMMEANS()
## set.wd()        Alpha()     PROCESS()
## import()        EFA()       model_summary()
## print_table()   CFA()       lavaan_summary()
##
## For full functionality, please install all dependencies:
## install.packages("bruceR", dep=TRUE)
##
## Online documentation:
## https://psychbruce.github.io/bruceR
##
## To use this package in publications, please cite:
## Bao, H. W. S. (2021). bruceR: Broadly useful convenient and efficient R functions (Version 2025.8) [Computer software]. https://doi.org/10.32614/CRAN.package.bruceR
##
##
## These packages are dependencies but not yet installed:
## - pacman, ggtext, see, lmtest, vars, phia, BayesFactor, GGally
##
## ***** Install All Dependencies *****
## install.packages("bruceR", dep=TRUE)
```

```
library(ggplot2)
library(scales)
```



```
##  
## Attaching package: 'scales'  
##  
## The following object is masked from 'package:purrr':  
##  
##     discard  
##  
## The following object is masked from 'package:readr':  
##  
##     col_factor
```

```
library(dbscan)
```

```
## Warning: package 'dbscan' was built under R version 4.5.2
```

```
##  
## Attaching package: 'dbscan'  
##  
## The following object is masked from 'package:stats':  
##  
##     as.dendrogram
```

```
library(cluster)  
library(scales)  
# daisy() for Gower
```

## 3.2 Import data: Monthly Advanced Metering Infrastructure (AMI) data

```
getwd()
```

```
## [1] "/Users/zhangtianlu/Library/CloudStorage/Box-Box/00 Tianlu Z/04 Internship/UT CON  
NECT & AW 2025/01 Syntax"
```

```
month_data <- read.csv("../02 Data/monthly_allocated_billing.csv")
```

### 3.3 Recoding aptment name & catogories

```

month_data <- month_data %>%
  mutate(property_name = case_when(
    SVC_ADDRESS == "2105 COMAL ST" ~ "UT_East_APT",
    SVC_ADDRESS == "2323 SAN ANTONIO ST" ~ "Castilian",
    SVC_ADDRESS == "2400 NUECES ST" ~ "2400_Nueces",
    SVC_ADDRESS == "2501 LAKE AUSTIN BLVD" ~ "Colorado_APT",
    SVC_ADDRESS == "2501 PEARL ST" ~ "The_Block_on_28th",
    SVC_ADDRESS == "2510 LEON ST" ~ "The_Block",
    SVC_ADDRESS == "2616 SALADO ST" ~ "Taxan",
    SVC_ADDRESS == "2819 RIO GRANDE ST" ~ "The_Block",
    SVC_ADDRESS == "505 W 22ND ST" ~ "The_Callaway_House",
    SVC_ADDRESS == "600 W 26TH ST" ~ "26_West",
    SVC_ADDRESS == "701 W 28TH ST" ~ "The_Block",
    SVC_ADDRESS == "702 W 25TH ST" ~ "The_Block",
    SVC_ADDRESS == "706 W MARTIN LUTHER KING JR BLVD" ~ "Crest_At_Pearl",
    SVC_ADDRESS == "904 W 22ND HALF ST" ~ "Vintage",
    SVC_ADDRESS == "2208 PEARL ST" ~ "The_Block",
    SVC_ADDRESS == "2200 PEARL ST" ~ "The_Block",
    SVC_ADDRESS == "2222 PEARL ST" ~ "The_Block",
    TRUE ~ NA_character_
  ))

month_data <- month_data %>%
  mutate(property_type = case_when(
    property_name %in% c("UT_East_APT", "Colorado_APT") ~ "Type2_Gradstudent_housing",
    property_name %in% c("Castilian", "The_Callaway_House") ~ "Type1_dorm_style",
    property_name %in% c("The_Block", "Vintage", "Taxan",
                        "26_West", "The_Block_on_28th", "Crest_At_Pearl") ~ "Type4_Large_Scale_APT",
    property_name %in% c("2400_Nueces") ~ "Type3_Shared_Occupancy_APT",
    TRUE ~ NA_character_
  ))

#month_data %>%
# select(SVC_ADDRESS, property_name, property_type) %>%
# View()

```

### 3.4 Filtered out irregration data

```

month_data_noirr <- month_data %>%
  filter(!grepl("^WI-", RS_CD))

nrow(month_data_noirr)

```

```
## [1] 21
```

```
## [1] 124
```

```
# try each apartment
month_data_noirr <- month_data_noirr %>%
  relocate(property_name, property_type, .after = SVC_ADDRESS)

# add usage for the same property
month_data_sum <- month_data_noirr %>%
  group_by(property_name, property_type) %>% # keep property_type
  summarise(
    n_rows = n(),
    SVC_ADDRESS = paste(sort(unique(SVC_ADDRESS)), collapse = " | "),
    across(starts_with("X"), ~ sum(.x, na.rm = TRUE)),
    .groups = "drop"
  )

#delete UT_East_APT because of too much missing values
month_data_sum <- month_data_sum %>%
  filter(property name != "UT East APT")
```

```
month_data_sum <- month_data_sum %>% select(-any_of("X"))

usage_long <- month_data_sum %>%
  pivot_longer(
    cols = starts_with("X"),
    names_to = c("month", "year"),
    names_pattern = "X(\\d{2})\\.\\. (\\d{4})\\.\\.usage",
    values_to = "usage"
  ) %>%
  mutate(
    month = as.integer(month),
    year = as.integer(year),
    date = ymd(sprintf("%04d-%02d-01", year, month)) # first day of month
  ) %>%
  arrange(property_name, date)

n_props <- length(unique(usage_long$property_name))
# 10 unique apartment names
```

## 3.7 Check & correct 0 usage referring hourly data

```

day_data <- read.csv("../02 Data/AMI_pbi_pull_duplicates_removed.csv")

day_data <- day_data %>%
  mutate(property_name = case_when(
    SVC_ADDRESS == "2105 COMAL ST" ~ "UT_East_APT",
    SVC_ADDRESS == "2323 SAN ANTONIO ST" ~ "Castilian",
    SVC_ADDRESS == "2400 NUECES ST" ~ "2400_Nueces",
    SVC_ADDRESS == "2501 LAKE AUSTIN BLVD" ~ "Colorado_APT",
    SVC_ADDRESS == "2501 PEARL ST" ~ "The_Block_on_28th",
    SVC_ADDRESS == "2510 LEON ST" ~ "The_Block",
    SVC_ADDRESS == "2616 SALADO ST" ~ "Taxan",
    SVC_ADDRESS == "2819 RIO GRANDE ST" ~ "The_Block",
    SVC_ADDRESS == "505 W 22ND ST" ~ "The_Callaway_House",
    SVC_ADDRESS == "600 W 26TH ST" ~ "26_West",
    SVC_ADDRESS == "701 W 28TH ST" ~ "The_Block",
    SVC_ADDRESS == "702 W 25TH ST" ~ "The_Block",
    SVC_ADDRESS == "706 W MARTIN LUTHER KING JR BLVD" ~ "Crest_At_Pearl",
    SVC_ADDRESS == "904 W 22ND HALF ST" ~ "Vintage",
    SVC_ADDRESS == "2208 PEARL ST" ~ "The_Block",
    SVC_ADDRESS == "2200 PEARL ST" ~ "The_Block",
    SVC_ADDRESS == "2222 PEARL ST" ~ "The_Block",
    TRUE ~ NA_character_
  ))

day_data <- day_data %>%
  mutate(property_type = case_when(
    property_name %in% c("UT_East_APT", "Colorado_APT") ~ "Type2_Gradstudent_housing",
    property_name %in% c("Castilian", "The_Callaway_House") ~ "Type1_dorm_style",
    property_name %in% c("The_Block", "Vintage", "Taxan",
                        "26_West", "The_Block_on_28th", "Crest_At_Pearl") ~ "Type4_Large_Scale_APT",
    property_name %in% c("2400_Nueces") ~ "Type3_Shared_Occupancy_APT",
    TRUE ~ NA_character_
  ))

RS_CD_data <- month_data %>%
  select(c("ACCT_ID", "RS_CD"))

day_data_merge <- day_data %>%
  left_join(RS_CD_data, by = "ACCT_ID") %>%
  relocate(RS_CD, property_type, .after = ACCT_ID)

```

```

## Warning in left_join(., RS_CD_data, by = "ACCT_ID"): Detected an unexpected many-to-many relationship between `x` and `y`.
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 3 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
## "many-to-many"` to silence this warning.

```

```
day_merge_fil <- day_data_merge %>%  
  filter(!grepl("^WI-", RS_CD))
```

## 3.8 Aggregating month & date

```
df <- day_merge_fil

# 1) Identify the usage column
usage_col <- grep("(?i)sum.*calculated.*consumption", names(df), value = TRUE)[1]

# 2) Make a pure date and month bucket
dat <- df %>%
  mutate(
    readingDate = as.Date(ReadingDate),
    month       = floor_date(readingDate, "month")
  )

# 3) Daily totals per property
daily_totals <- dat %>%
  group_by(property_name, readingDate) %>%
  summarise(
    n_rows = n(),
    SVC_ADDRESS = paste(sort(unique(SVC_ADDRESS)), collapse = " | "),
    usage_gal = sum(.data[[usage_col]], na.rm = TRUE),
    .groups = "drop"
  )

# 4) Monthly totals per property
monthly_totals <- dat %>%
  group_by(property_name, month) %>%
  summarise(
    n_rows = n(),
    SVC_ADDRESS = paste(sort(unique(SVC_ADDRESS)), collapse = " | "),
    usage_gal = sum(.data[[usage_col]], na.rm = TRUE),
    .groups = "drop"
  )

lookup <- monthly_totals %>%
  filter(month == as.Date("2025-06-01"),
         property_name %in% c("26_West", "The_Block_on_28th", "The_Callaway_House")) %>%
  rename(date = month) %>% # change column name
  select(property_name, date, usage_gal)

# 2) Replace usage in usage_long for June 1, 2025 with the monthly total
usage_long <- usage_long %>%
  left_join(lookup, by = c("property_name", "date")) %>%
  mutate(
    usage_update = coalesce(usage_gal, usage)
  )

## other 0 coded as missing
usage_long <- usage_long %>%
  mutate(
```

```
usage_update = na_if(usage_update, 0)
)
```

## 4.Descriptive analysis

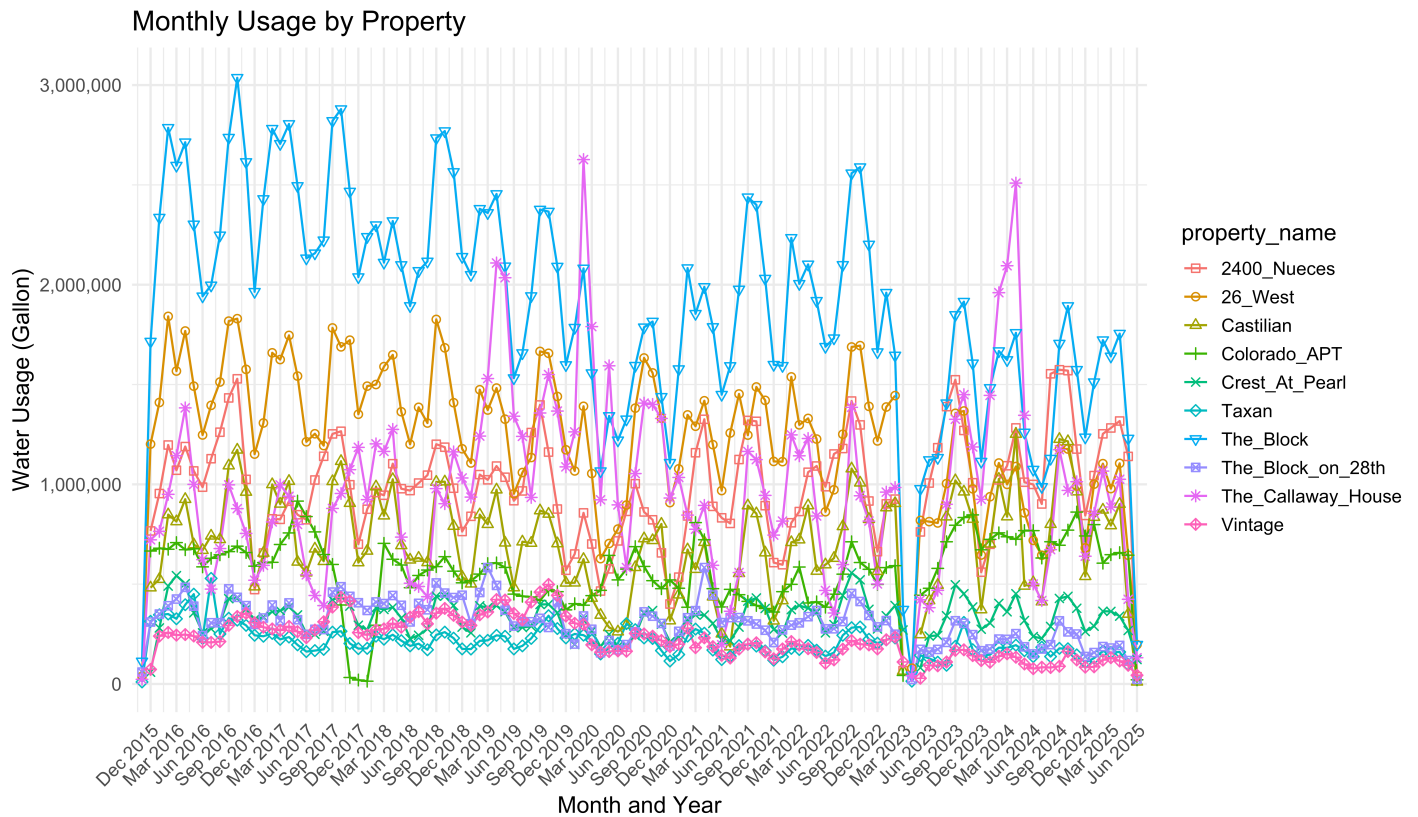
### 4.1 Visualize the trend for each student housing

```
shape_values <- rep(c(0:25), length.out = n_props) # at least n_props shapes

ggplot(usage_long, aes(x = date, y = usage_update, color = property_name)) +
  geom_line() +
  geom_point(aes(shape = property_name)) +
  scale_shape_manual(values = shape_values) + # make sure all have unique shapes
  scale_x_date(
    date_breaks = "3 months",
    date_labels = "%b %Y",
    expand = c(0.01, 0.01)
  ) +
  scale_y_continuous(labels = comma) +
  labs(
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    title = "Monthly Usage by Property"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 15 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



## 4.2 Descriptive & Visualize for each category of student housing

```
type_long_update <- usage_long %>%
  mutate(date = as.Date(date)) %>% # ensure Date type
  group_by(property_type, date) %>%
  summarise(
    SVC_ADDRESS = paste(sort(unique(property_name)), collapse = " | "), # <- added
    n_rows = n(),
    n_nonmissing = sum(!is.na(usage_update)),
    usage_update_sum = sum(usage_update, na.rm = TRUE),
    usage_update_mean = if_else(n_nonmissing > 0,
                                usage_update_sum / n_nonmissing,
                                NA_real_),
    .groups = "drop"
  )
```



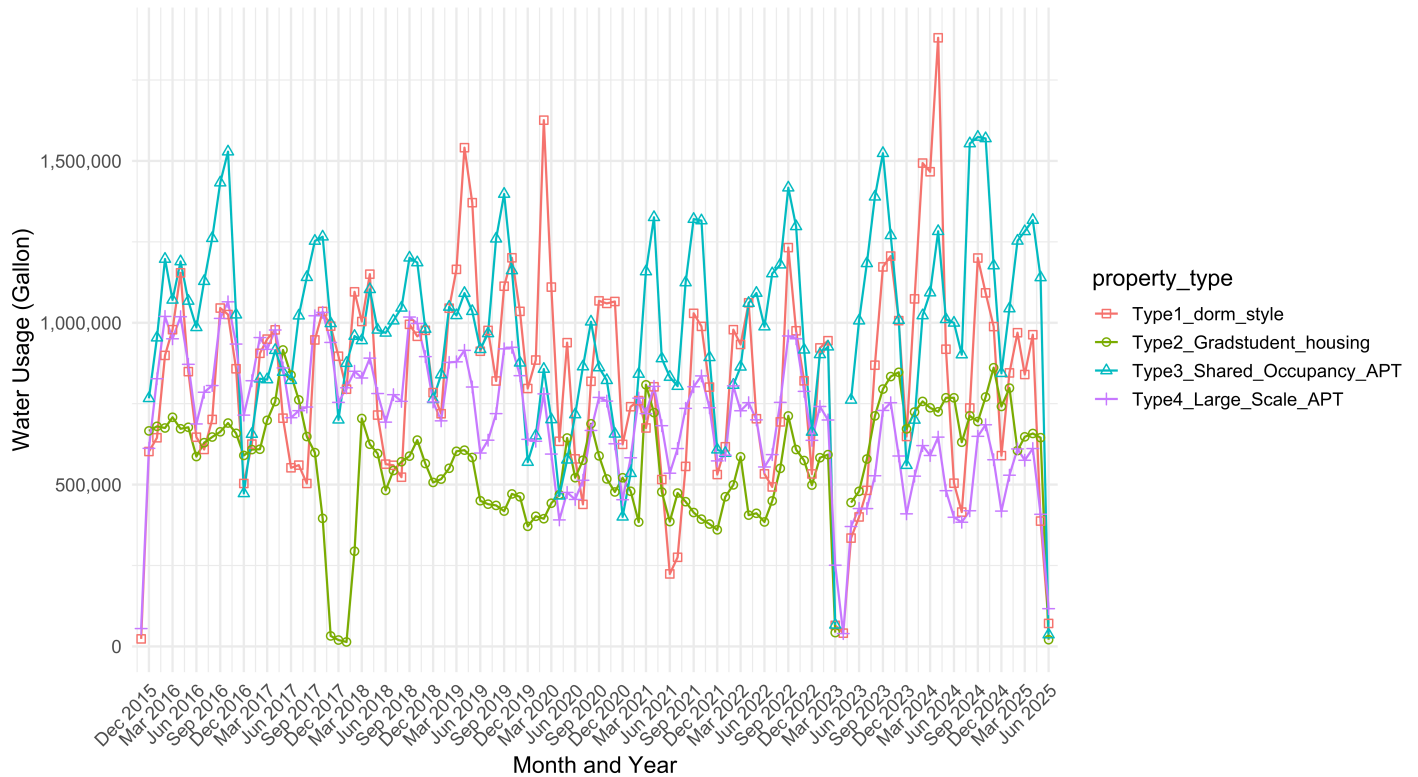
```
# Pick distinct shapes (see ?pch for full list; avoid 0-14 if you want filled symbols)
shape_values <- rep(c(0:25), length.out = n_props) # at least n_props shapes

ggplot(type_long_update, aes(x = date, y = usage_update_mean, color = property_type)) +
  geom_line() +
  geom_point(aes(shape = property_type)) +
  scale_shape_manual(values = shape_values) +
  scale_x_date(
    date_breaks = "3 months",
    date_labels = "%b %Y",
    expand = c(0.01, 0.01)
  ) +
  scale_y_continuous(labels = comma) +
  labs(
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    title = "Monthly Usage by Property Type"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

## Monthly Usage by Property Type



## 5. Main analysis: Clustering using Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

```
##calculate date
type_long_update <- type_long_update %>%
  mutate(date_num = as.numeric(date - min(date, na.rm = TRUE))) # days since earliest date
```

## 5.1 Type1:dorm\_style

```
## DBSCAN * KNN
type1_data <- type_long_update %>%
  filter(property_type == "Type1_dorm_style")

type1_data_db <- type1_data %>%
  select(c("usage_update_mean"))

k <- 5
kNNdist_vals <- kNNdist(type1_data_db, k = k)

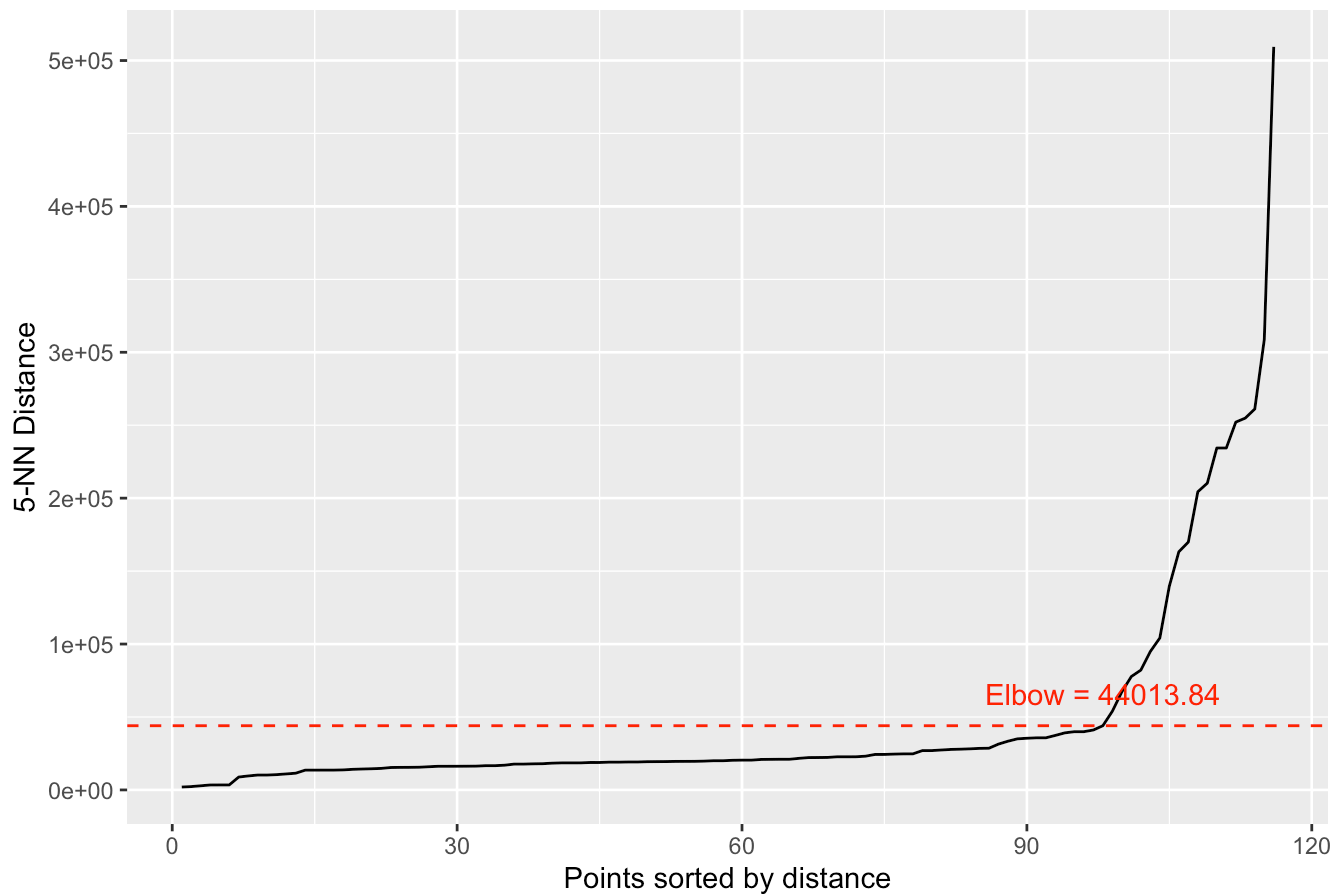
# Sort distances
sorted_dist <- sort(kNNdist_vals)

# --- Elbow detection (max distance to line) ---
p1 <- c(1, sorted_dist[1])
p2 <- c(length(sorted_dist), sorted_dist[length(sorted_dist)])
dist_to_line <- sapply(1:length(sorted_dist), function(i) {
  num <- abs((p2[2] - p1[2]) * i - (p2[1] - p1[1]) * sorted_dist[i] + p2[1]*p1[2] - p2[2]*p1[1])
  den <- sqrt((p2[2] - p1[2])^2 + (p2[1] - p1[1])^2)
  num / den
})
elbow_idx <- which.max(dist_to_line)
elbow_val <- sorted_dist[elbow_idx]

# --- Plot with elbow line ---
qplot(y = sorted_dist, x = 1:length(sorted_dist), geom = "line") +
  geom_hline(yintercept = elbow_val, color = "red", linetype = "dashed") +
  labs(
    title = "k-NN Distance Plot for DBSCAN eps Selection",
    x = "Points sorted by distance",
    y = paste0(k, "-NN Distance")
  ) +
  annotate("text", x = elbow_idx, y = elbow_val, label = paste("Elbow =", round(elbow_val, 2)),
    vjust = -1, color = "red")
```

```
## Warning: `qplot()` was deprecated in ggplot2 3.4.0.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

## k-NN Distance Plot for DBSCAN eps Selection



```
min_pts <- 10    # You can also adjust

db_type1 <- dbscan(type1_data_db, eps = elbow_val, minPts = min_pts)

# Append results
type1_data$cluster <- db_type1$cluster

Freq(type1_data$cluster)
```

```
## Frequency Statistics:
```

```
## _____
##      N      %
## _____
## 0    27 23.3
## 1    32 27.6
## 2    57 49.1
## _____
## Total N = 116
```

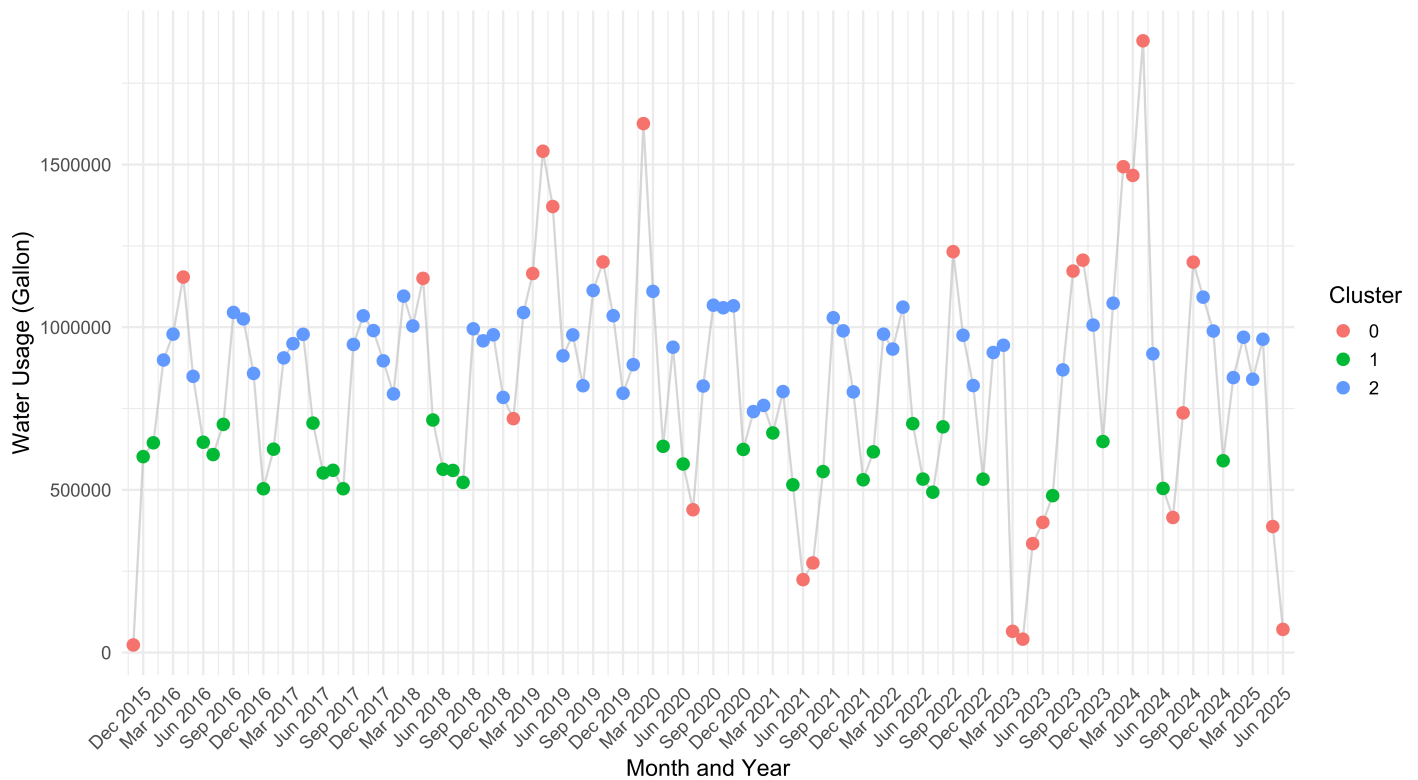
## ## Visualization

```

ggplot(type1_data, aes(x = date, y = usage_update_mean, color = as.factor(cluster))) +
  geom_line(aes(group = 1), color = "grey70", alpha = 0.5) + # subtle grey line
  geom_point(size = 3) +
  scale_x_date(
    date_labels = "%b %Y",    # e.g., Jan 2015
    date_breaks = "3 months", # show a tick every 3 months
    expand = c(0.01, 0.01)
  ) +
  labs(
    title = "Type1 Dorm style",
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    color = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1) # rotate labels
  )

```

Type1 Dorm style



## 5.2 Type2\_Gradstudent\_housing

```
## DBSCAN * KNN
type2_data <- type_long_update %>%
  filter(property_type == "Type2_Gradstudent_housing") %>%
  mutate(date = as.Date(date))

features <- c("usage_update_mean") # features used for kNN/DBSCAN
k <- 5                               # k for k-NN distance plot
min_pts <- 10                        # DBSCAN minPts

# --- Listwise deletion on the features; keep row id to map back ---
t2_clean <- type2_data %>%
  mutate(.row_id = dplyr::row_number()) %>%
  select(.row_id, dplyr::all_of(features)) %>%
  mutate(dplyr::across(dplyr::all_of(features), as.numeric)) %>%
  filter(dplyr::if_all(dplyr::all_of(features), ~ is.finite()))

# Safety check
stopifnot(nrow(t2_clean) >= max(k, min_pts))

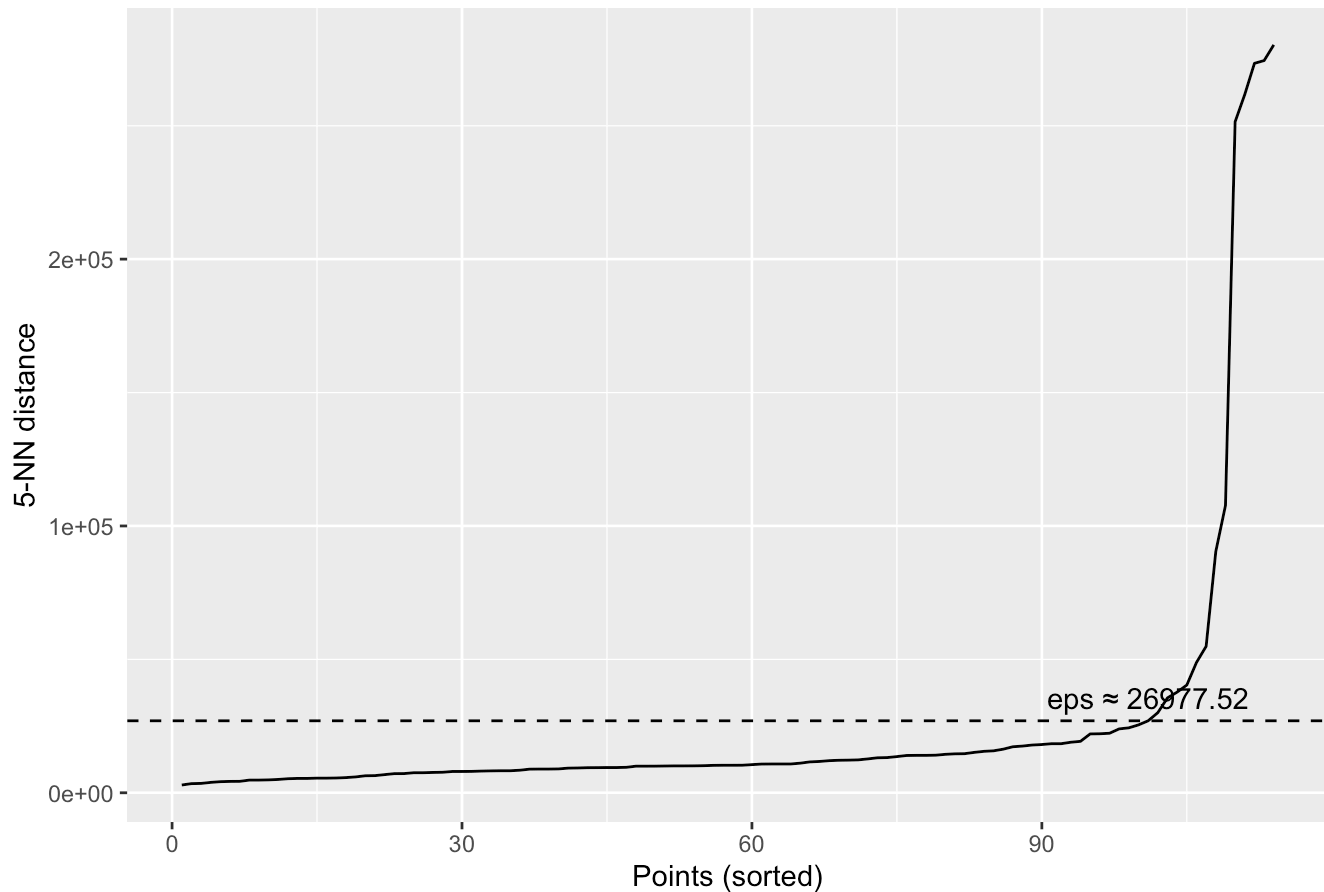
# --- Numeric matrix with no NA/Inf for kNN/DBSCAN ---
X <- as.matrix(t2_clean[features])

# --- k-NN distances & elbow detection ---
kNNdist_vals <- kNNdist(X, k = k)
sorted_dist <- sort(as.vector(kNNdist_vals))
n <- length(sorted_dist)

p1 <- c(1, sorted_dist[1])
p2 <- c(n, sorted_dist[n])
den <- sqrt((p2[2]-p1[2])^2 + (p2[1]-p1[1])^2)
dist_to_line <- abs((p2[2]-p1[2])*(1:n) - (p2[1]-p1[1])*sorted_dist + p2[1]*p1[2] - p2[2]*p1[1]) / den
elbow_idx <- which.max(dist_to_line)
eps_auto <- sorted_dist[elbow_idx]

# Plot
ggplot(data.frame(i = 1:n, d = sorted_dist), aes(i, d)) +
  geom_line() +
  geom_hline(yintercept = eps_auto, linetype = "dashed") +
  annotate("text", x = elbow_idx, y = eps_auto,
    label = paste("eps \u2248", round(eps_auto, 2)), vjust = -0.6) +
  labs(title = paste0(k, "-NN Distance Plot"),
    x = "Points (sorted)", y = paste0(k, "-NN distance"))
```

## 5-NN Distance Plot



```
# --- DBSCAN on the cleaned matrix ---
db_type2 <- dbSCAN(X, eps = eps_auto, minPts = min_pts)

# --- Map clusters back to the full (pre-clean) data ---
type2_data$cluster <- NA_integer_
type2_data$cluster[t2_clean$.row_id] <- db_type2$cluster

# Cluster counts (noise is 0)
table(type2_data$cluster, useNA = "ifany")
```

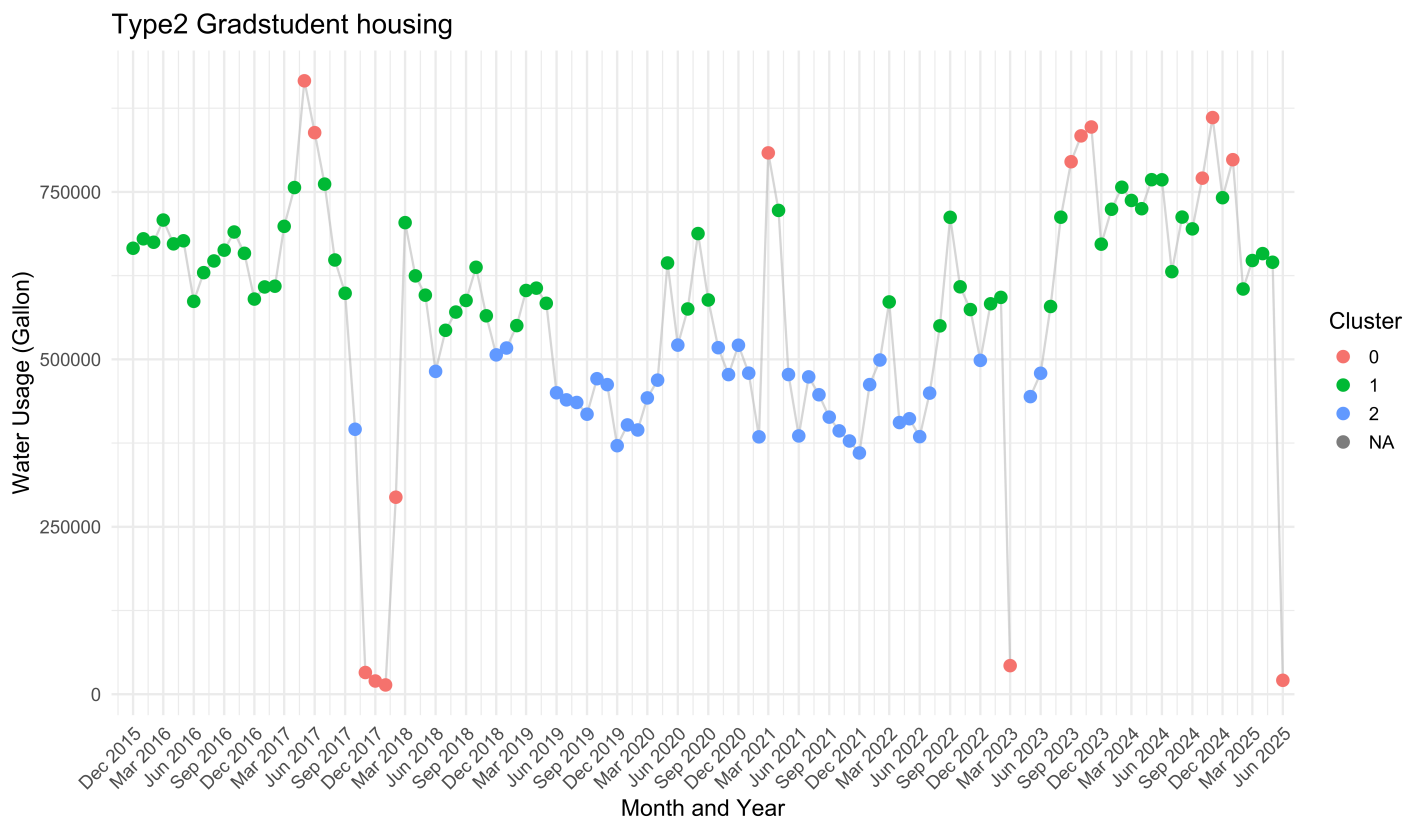
```
##
##      0      1      2 <NA>
##    15     61     38      2
```

## ## Visualization

```
ggplot(type2_data, aes(x = date, y = usage_update_mean, color = as.factor(cluster))) +
  geom_line(aes(group = 1), color = "grey70", alpha = 0.5) +
  geom_point(size = 3) +
  scale_x_date(
    date_labels = "%b %Y",    # e.g., Jan 2015
    date_breaks = "3 months", # show a tick every 3 months
    expand = c(0.01, 0.01)
  ) +
  labs(
    title = "Type2 Gradstudent housing",
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    color = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1) # rotate labels
  )
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).
```





## 5.3 Type3\_Shared\_Occupancy\_APT

```
## DBSCAN * KNN
type3_data <- type_long_update %>%
  filter(property_type == "Type3_Shared_Occupancy_APT") %>%
  mutate(date = as.Date(date))

# ----- settings -----
features <- c("usage_update_mean") # the feature(s) for DBSCAN
k <- 5                               # for k-NN distance
min_pts <- 8                         # DBSCAN minPts

# ----- listwise deletion on the features -----
t3_clean <- type3_data %>%
  mutate(.row_id = row_number()) %>%
  select(.row_id, all_of(features)) %>%
  mutate(across(all_of(features), as.numeric)) %>%
  filter(if_all(all_of(features), ~ is.finite(.))) # drop rows with NA/Inf

# safety check
if (nrow(t3_clean) < max(k, min_pts)) {
  stop("Not enough non-missing rows after cleaning for chosen k/minPts.")
}

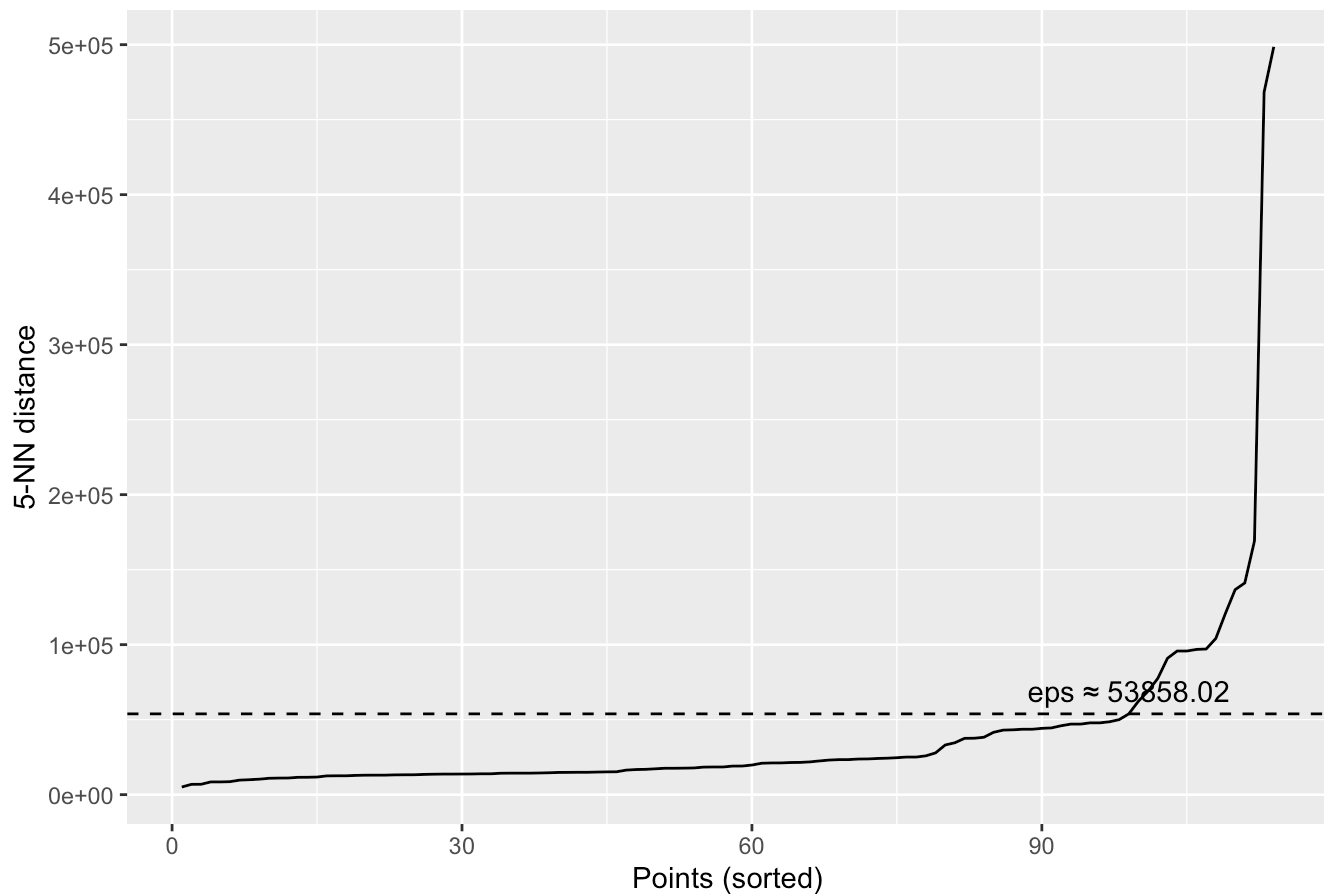
# matrix for kNN/DBSCAN
X <- as.matrix(t3_clean[features])

# ----- k-NN distances & elbow eps -----
kNNdist_vals <- kNNdist(X, k = k)
sorted_dist <- sort(as.vector(kNNdist_vals))
n <- length(sorted_dist)

p1 <- c(1, sorted_dist[1])
p2 <- c(n, sorted_dist[n])
den <- sqrt((p2[2] - p1[2])^2 + (p2[1] - p1[1])^2)
dist_to_line <- abs((p2[2] - p1[2]) * (1:n) - (p2[1] - p1[1]) * sorted_dist + p2[1]*p1
[2] - p2[2]*p1[1]) / den
elbow_idx <- which.max(dist_to_line)
eps_auto <- sorted_dist[elbow_idx]

# visualize (optional)
ggplot(data.frame(i = 1:n, d = sorted_dist), aes(i, d)) +
  geom_line() +
  geom_hline(yintercept = eps_auto, linetype = "dashed") +
  annotate("text", x = elbow_idx, y = eps_auto, label = paste("eps ≈", round(eps_auto,
2)), vjust = -0.6) +
  labs(title = paste0(k, "-NN Distance Plot"), x = "Points (sorted)", y = paste0(k, "-NN
distance"))
```

## 5-NN Distance Plot



```
# ---- DBSCAN on the cleaned matrix ----
db <- dbscan(X, eps = eps_auto, minPts = min_pts)

# map clusters back to the full (pre-clean) data
type3_data$cluster <- NA_integer_
type3_data$cluster[t3_clean$.row_id] <- db$cluster

# cluster counts (noise is 0)
table(type3_data$cluster, useNA = "ifany")
```

```
##
##      0      1      2 <NA>
##    15     86     13      2
```

## ## Visualization

```

ggplot(type3_data, aes(x = date, y = usage_update_mean, color = as.factor(cluster))) +
  geom_line(aes(group = 1), color = "grey70", alpha = 0.5) +
  geom_point(size = 3) +
  scale_x_date(
    date_labels = "%b %Y",    # e.g., Jan 2015
    date_breaks = "3 months", # show a tick every 3 months
    expand = c(0.01, 0.01)
  ) +
  labs(
    title = "Type3 Shared Occupancy APT",
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    color = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1) # rotate labels
  )

```

```

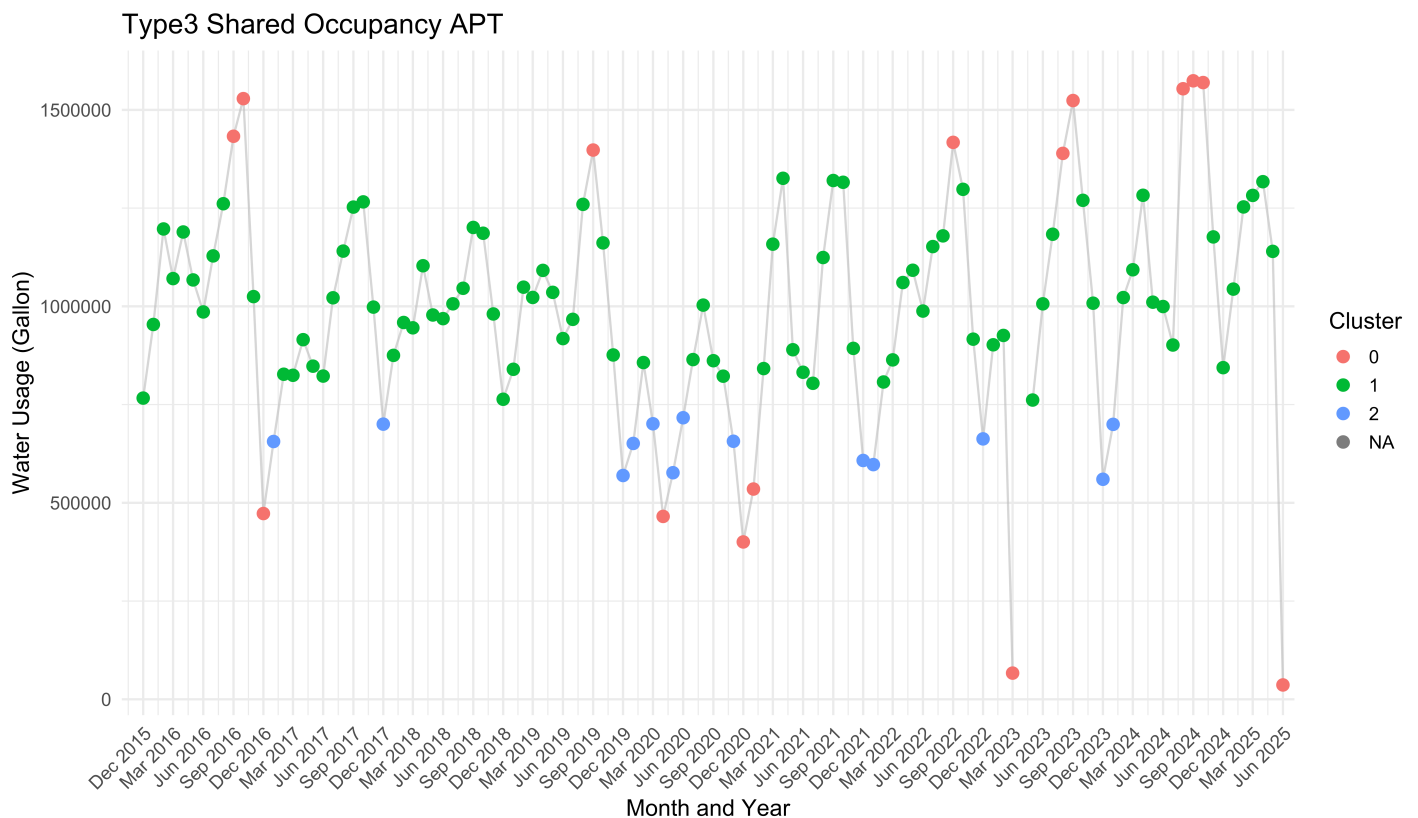
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

```

```

## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).

```



## 5.4 Type4\_Large\_Scale\_APT

```
## DBSCAN * KNN
type4_data <- type_long_update %>%
  filter(property_type == "Type4_Large_Scale_APT")

type4_data_db <- type4_data %>%
  select(c("usage_update_mean"))

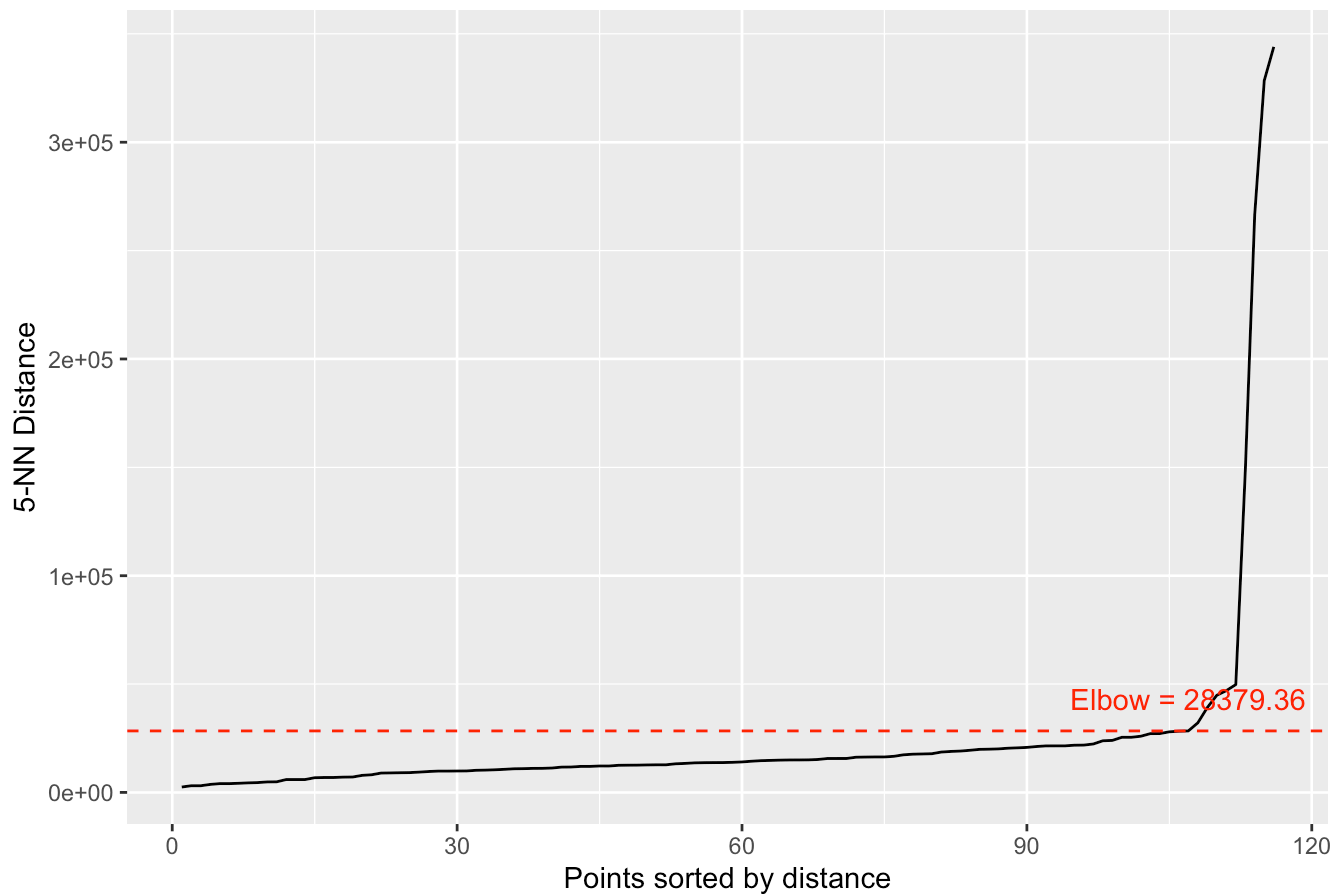
k <- 5
kNNdist_vals <- kNNdist(type4_data_db, k = k)

# Sort distances
sorted_dist <- sort(kNNdist_vals)

# --- Elbow detection (max distance to line) ---
p1 <- c(1, sorted_dist[1])
p2 <- c(length(sorted_dist), sorted_dist[length(sorted_dist)])
dist_to_line <- sapply(1:length(sorted_dist), function(i) {
  num <- abs((p2[2] - p1[2]) * i - (p2[1] - p1[1]) * sorted_dist[i] + p2[1]*p1[2] - p2[2]*p1[1])
  den <- sqrt((p2[2] - p1[2])^2 + (p2[1] - p1[1])^2)
  num / den
})
elbow_idx <- which.max(dist_to_line)
elbow_val <- sorted_dist[elbow_idx]

# --- Plot with elbow line ---
qplot(y = sorted_dist, x = 1:length(sorted_dist), geom = "line") +
  geom_hline(yintercept = elbow_val, color = "red", linetype = "dashed") +
  labs(
    title = "k-NN Distance Plot for DBSCAN eps Selection",
    x = "Points sorted by distance",
    y = paste0(k, "-NN Distance")
  ) +
  annotate("text", x = elbow_idx, y = elbow_val, label = paste("Elbow =", round(elbow_val, 2)),
    vjust = -1, color = "red")
```

## k-NN Distance Plot for DBSCAN eps Selection



```
min_pts <- 10

db_type4 <- dbscan(type4_data_db, eps = elbow_val, minPts = min_pts)

# Append results
type4_data$cluster <- db_type4$cluster

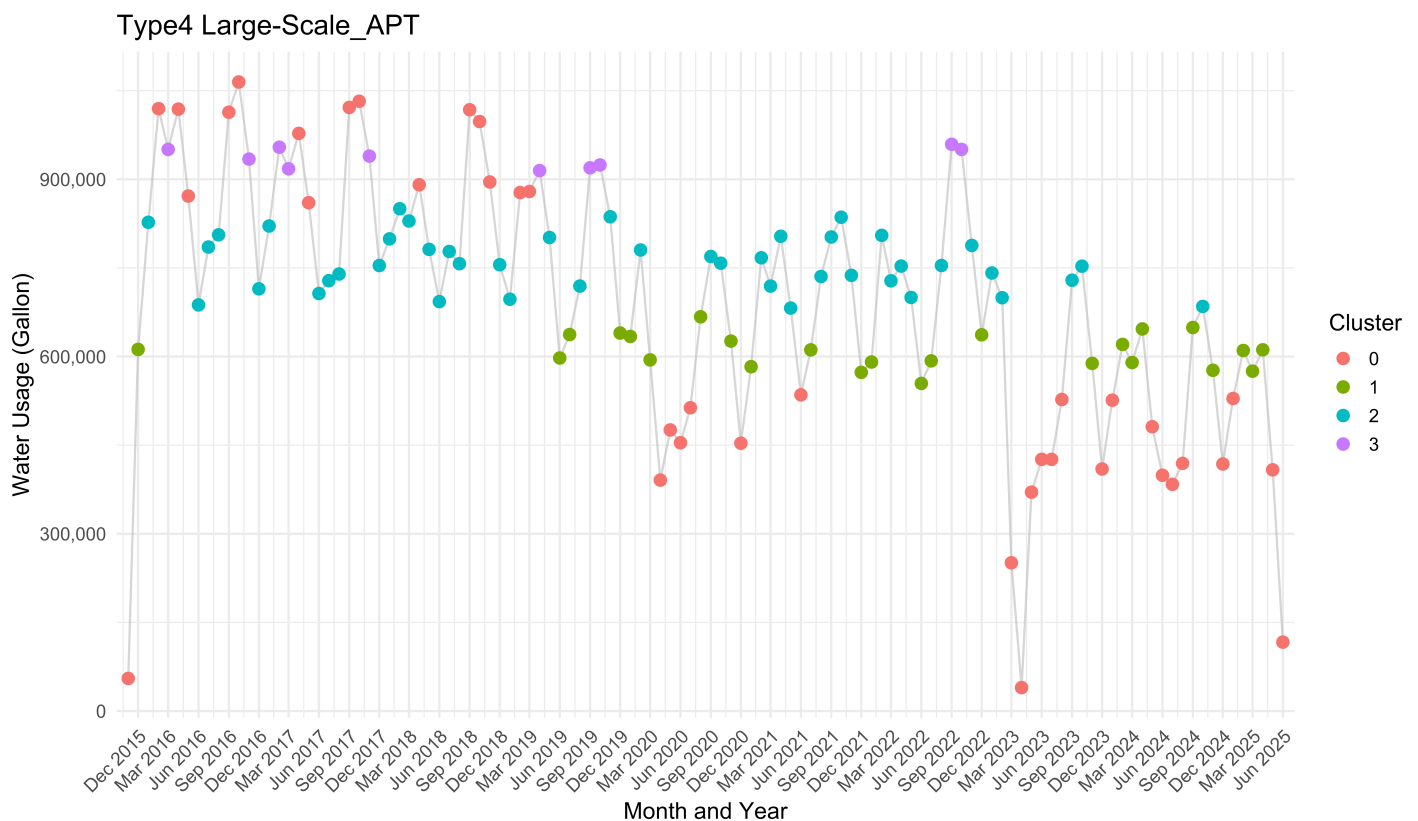
Freq(type4_data$cluster)
```

```
## Frequency Statistics:
```

```
## _____
##      N      %
## _____
## 0   38  32.8
## 1   24  20.7
## 2   44  37.9
## 3   10   8.6
## _____
## Total N = 116
```

### ## Visualization

```
ggplot(type4_data, aes(x = date, y = usage_update_mean, color = as.factor(cluster))) +
  geom_line(aes(group = 1), color = "grey70", alpha = 0.5) +
  geom_point(size = 3) +
  scale_x_date(
    date_labels = "%b %Y",    # e.g., Jan 2015
    date_breaks = "3 months", # show a tick every 3 months
    expand = c(0.01, 0.01)
  ) +
  scale_y_continuous(labels = comma) +
  labs(
    title = "Type4 Large-Scale_APT",
    x = "Month and Year",
    y = "Water Usage (Gallon)",
    color = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1) # rotate labels
  )
```



## Overall conclusion again

**Different types of student housing shows different water usage patterns through 2015 Dec until 2025** Type 1: Dorm-style housing: Shows pronounced seasonal variation, with clear increases during the summer months (May–August). Type 2: Graduate student housing: Displays relatively stable usage throughout the year, with less dramatic seasonal fluctuation. However, it appears more sensitive to external disruptions, such as COVID-19.

Type 3: Shared-occupancy apartments: Characterized by lower water usage primarily during the winter months (December–January), with minimal variation across other seasons. Type 4: Large-scale apartment complexes: Demonstrates a decline in water usage during and after the COVID-19 period, particularly after 2023. Seasonal patterns are still evident, with higher usage in summer (May–August) and a smaller dip in winter (December).

## 6. Bibliography

Chen, Y., Zhou, L., Pei, S., Yu, Z., Chen, Y., Liu, X., Du, J., & Xiong, N. (2021). KNN-BLOCK DBSCAN: Fast Clustering for Large-Scale Data. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 51(6), 3939–3953. <https://doi.org/10.1109/TSMC.2019.2956527> (<https://doi.org/10.1109/TSMC.2019.2956527>)

Ester, M., Kriegel, H.-P., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.

Fu, H., Lee, S., Baltazar, J.-C., & Claridge, D. E. (2021). Use of Domestic Water Consumption as Proxy for Occupancy Level in Building Cooling Energy Model. 127.

Hahsler, M., Piekenbrock, M., & Doran, D. (2019). dbscan: Fast Density-Based Clustering with R. *Journal of Statistical Software*, 91, 1–30. <https://doi.org/10.18637/jss.v091.i01> (<https://doi.org/10.18637/jss.v091.i01>)